Python and Git Class Assignments Setup Tutorial

East Texas A&M University - Commerce

Summer 2026

Getting Started with Python and Git for Class Assignments

Programming environment and assignment configuration instructions for East Texas A&M University Computer Science and Information System classes. The development environment you set up here may be used for many different classes, such as application program development, data structures, introduction to AI with Python, etc., for both undergraduate and graduate ETAMU Science and Engineering classes.

In this tutorial you will learn how to setup Python 3 on your own computing system, as well as install and setup `git` tools and configure `ssh` key access to a GitHub account. Python is used as a language for assignments in many classes, and you may be using GitHub classrooms in your class to accept, work on and submit assignments for grading.

Prerequisites

You will need access to a computer or laptop that you can install software on. Any relatively recent hardware with Windows 10/11, MacOS X, or Linux operating systems should be able to work. You cannot usually use Android, iOS or Chrome Book systems. For Macintosh systems, both Intel based hardware and the newer M1/M2/M3/M4 Apple silicon hardware should work. For Windows and Linux systems, Intel/AMD systems or newer ARM CPUs should also be fine.

The resource requirements to run `python` scripts and use `git` tools are moderate. We recommend a system with 4GB of memory and maybe 5+GB of free disk space be available to install and use these tools.

Install a Python Interpreter

You will need a Python 3 interpreter installed and available to be run from a command line in your system `PATH` to work on assignments for this class. There are many different ways to install suitable Python interpreters on your Windows, MacOS or Linux system.

For this class you should not use any Python 3 version that is past or close to end-of-life. As of when these instructions are being written, that means you should use Python 3.14, though if you already have 3.11 through 3.13 those will probably be fine for this class. See the python.org site for information about current Python versions.

Windows

For Windows OS, if you do not already have a Python distribution installed, it is recommended that you download the most recent distribution directly from python.org.

Get the most recent installer for Python 3 and run the installer on your Windows system. The standard installer on Windows will pop up a dialog asking if you want to install `cpython` which you can install but probably won't need for this class.

Once the installer finishes, check that you have the Python interpreter available. If you open your Start menu you should now find that you have application buttons to start a Python interpreter, the Python IDLE environment, and the Python install manager. As of this tutorial, you should be using Python 3.14 or newer.

You should check that you have your Python interpreter available from your system `PATH` in a command terminal. If you don't know how to open a command prompt, see How to open a Command Prompt in Windows 10, 11. Open a command prompt and check that the python interpreter is in your path, and that you can run it and confirm you are running version 3.14 like this:

```
C:\Users\etamustudent> where python
C:\Users\etamustudent\AppData\Local\Python\bin\python.exe

C:\Users\etamustudent> python --version
Python 3.14.6
```

This shows running a terminal session logged into my windows system as a user named `etamustudent`. The `where` command shows that there is a `python.exe` interpreter installed and on my path in my `AppData\Local` directory. And running `python --version` shows that we can actually run the `python` interpreter, and that we have version 3.14.6 currently installed.

MacOS

MacOS systems probably have a system version of Python already installed, but it is often not the version you want to use for these class assignment. It is recommended that you install and use the Homebrew macOS package management system to install python 3. You will need to How to use the Terminal command line in MacOS.

You can read Installing Python 3 on Mac OS X for details on installing Python 3 on MacOS. Following those instructions, you would perform the following commands in a Terminal window to install Xcode, Homebrew and Python 3

TODO: I haven't actually run these on a real MacOS system, need to confirm and correct actual command line commands needed here.

```
$ xcode-select --install
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master/install.sh)"
$ brew install python
```

After you think you have a Python interpreter installed, check that it is available from a Terminal like this:

```
$ which python
/Users/etamuuser/bin/python

$ python --version
Python 3.14.6
```

Linux

The system default install of Python will usually work fine for this classes assignments. Test that Python is available by opening a Terminal and performing:

```
$ which python3
/usr/bin/python3

$ python3 --version
Python 3.14.6
```

Though note as shown here there may not be an executable named simply `python` in your `PATH` on linux systems, so you may need to refer to the interpreter as `python3` as shown here.

If there does not appear to be a suitable Python interpreter on your Linux system, use your systems package manager (`apt` on Debian / Ubuntu systems, `yum` on CentOS / Red hat systems). For example, on Debian / Ubuntu systems, you would usually install `python3` packages like this:

```
$ sudo apt install python3
```

Post Python Install Checks

For all operating systems, you should have a Python 3 interpreter available on your command line `PATH`. Open up a terminal or command prompt on your system and perform the following to check that the Python interpreter is available.

```
# Windows systems
$ where python

# Linux or MacOs systems
$ which python
```

The command in a Windows command prompt is named `where` but it is called `which` on Linux / MacOS systems. If a Python interpreter is installed and available on your system `PATH`, the full path to the executable will be displayed. If it is not installed, nothing will be displayed when running these commands.

Note: On MacOS and Linux systems, depending on how you install Python, there may not be a command named `python` and you will instead need to use `python3` to refer to and run the Python 3 interpreter.

If a Python interpreter is available on your system `PATH`, you can check which version you have by running:

```
$ python --version
3.14.6
```

Or on some systems, as mentioned, you need to use `python3` instead. For assignments in this class, you probably should not be using a version older than 3.11, and as of the writing of this guide version 3.14.6 is the most recent Python 3 version available.

Install `git` and `ssh` Tools

In addition to a Python 3 interpreter, you may be required to have the `git` revision control tools available on your system to create and submit assignments with.

Windows

Windows OS does not come preinstalled with `git` nor `ssh` tools. You should install these by installing the official Git - Install for Windows package.

Download the correct installer from that link. The 'Click here to download' link, I believe, autodetects whether you have an x86/amd system or an ARM64 system and gives you the correct version of the installer. If not check that you are using the right version for your hardware system.

When you run the `git` installer, it will ask you many questions about how you want the tools installed. You should accept all of the defaults, unless you are familiar with special needs you have, except for the following:

- You should override the default branch name for new repositories to use 'main' as your default branch name.
- The default should be to install and "Use bundled OpenSSH". Ensure it is using the bundled OpenSSH as you probably do not already have `ssh` tools installed on your system.
- It is best to configure your line ending conversions to be "Checkout as-is, commit as-is", as converting to windows-style line endings can cause problems for some assignments.

Once `git` and the OpenSSH `ssh` tools have been installed, test they are available from a normal Command Prompt, see the general section below on testing and configuring `git` and `ssh`.

MacOS

You can install from the official Git - Install for MacOS.

Or, if you already set up `Homebrew` to install Python 3, an often better and preferred method is to have `Homebrew` install your git tools.

```
$ brew install git
```

Proceed to testing and configuring `git` and `ssh` below once you have a working set of `git` tools installed.

Linux

`git` and `ssh` tools will most likely already be installed and available on your system, see below for testing and configuring `git` and `ssh`.

If `git` is not installed on your system, use your package manager to install them. For example, for Debian / Ubuntu systems that use the `apt` package manager, there is usually a `build-essential` package that installs `git` along with other basic programming build tools:

```
$ sudo apt install build-essential
```

'git' and 'ssh' Post Install and Configuration

You can test that you have good versions of the `git` and `ssh` tools installed on your system using the `which` / `where` command, and checking which version is available.

On Windows systems, use `where` and check the version of `git` and `ssh` like this:

```
C:\Users\etamustudent> where git
C:\Program Files\Git\cmd\git.exe

C:\Users\etamustudent> git --version
git version 2.55.0.windows.2

C:\Users\etamustudent>where ssh
C:\Windows\System32\OpenSSH\ssh.exe

C:\Users\etamustudent>ssh -V
OpenSSH_for_Windows_9.5p1, LibreSSL 3.8.2
```

Note That is a capital `-V` not lowercase. Case matters when specifying command line flags, the `-v` option is different from the `-V` option for the `ssh` command.

On MacOS and Linux systems, use the `which` command.

```
$ which git
/usr/bin/git

$ git --version
git version 2.53.0

$ which ssh
/usr/bin/ssh

$ ssh -V
OpenSSH_10.2p1 Ubuntu-2ubuntu3.2, OpenSSL 3.5.5 27 Jan 2026
```

On all systems, you should set basic `git` global configuration settings at this point once you have a working `git` tool installed. See Customizing Git - Git Configuration for a more detailed discussion. In general, for our class assignments, you have to at a minimum configure your user name and email correctly. Perform the following from a Terminal to set basic git global configuration settings:

```
C:\Users\etamustudent> git config --global user.name "ETAMU Student"

C:\Users\etamustudent> git config --global user.email "estudent@leomail.etamu.edu"

C:\Users\etamustudent> git config --list
...
```

```
user.name=ETAMU Student
user.email=estudent@leomail.etamu.edu
```

You can list the current global git configurations as shown using the `--list` flag. Other configuration settings may be set besides your `user.name` and `user.email`, but do check that those two values were correctly set in your configuration here.

Note: Of course you should set your name here for the `user.name`.

Note: The email address you configure here needs to be the same as the e-mail address that you use as the primary e-mail for your GitHub account, which is discussed next. If you are not using your school Leo Mail e-mail for GitHub, set the `user.email` config here appropriately.

Create GitHub Account and Set Up `ssh` key

In order to use GitHub classrooms for assignment submissions and grading, you will need to create a GitHub account for yourself. If you don't already have one, go to GitHub.com and create a free account.

If you are creating a new account, you can use your Leo Mail e-mail address as your primary e-mail contact, though it is usually fine to use a different e-mail address for our classes. But make sure that whatever is your primary e-mail address on your GitHub account, that you are using the same e-mail for your `user.email` global git configuration setting.

Once you have a working GitHub account, you need to generate an `ssh` public/private key pair and add the public key to your GitHub account settings. You can read the GitHub Instructions for Adding a new SSH Key to your GitHub Account for additional information on performing the next steps.

For all operating systems, if you have `ssh` tools installed, you should be able to do the following to generate a new default `ssh` key:

```
C:\Users\etamustudent> ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\etamustudent/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\etamustudent/.ssh/id_ed25519
Your public key has been saved in C:\Users\etamustudent/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:LeDfO85S+F4HGAhzD73pikAAVHlZYgUGlbUYWdiqI4s etamustudent@windemo
The key's randomart image is:
+--[ED25519 256]--+
|oo.o+B#=o.       |
|  ..o*+=.+.      |
|   ...o.. oo     |
|    .o . .oo     |
|   .. . Soo .    |
| . o.  ..oo  .   |
|. o .. ..+. . .  |
|E.    . o.oo .   |
|         +=.     |
+----[SHA256]-----+
```

The example shown here was performed on a Windows system, but you should have the `ssh-keygen` tool available on any system with `ssh` installed on it.

Note: As shown the private key was saved into a directory named `c:\Users\etamustudent\.ssh\id_ed25519` and the public key is in the same directory, but just named `id_ed25519.pub`. The user name we are using in ths example is `etamustudent`. On a MacOS system, your keys will be placed in your `/Users/etamustudent/.ssh` directory, and on Linux systems the default location is `/home/etamustudent/.ssh` You need to know the location of the `id_ed25519.pub` public key in order to copy and paste it to your GitHub account.

Note: You should not enter in any passphrase for your new key, unless you are familiar with using `ssh` keys and passphrases for them in general. A passphrase adds an additional layer of security to your key, but is not necessary for our keys for our assignments, and it will be easier to use if you are just learning how to work with `ssh` keys if you don't add a passphrase.

Once you have a default key generated, the public key information will be in the `.ssh/id_ed25519.pub` file from your home directory. Open this file with a text editor and copy this public key. Alternatively you can display the key using the `type` command on Windows systems, or the `cat` command on MacOS / Linux systems:

```
# on Windows
C:\Users\etamustudent> type .ssh\id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDdcar+W6ILPAXtBW6xfIH5cpVHwR2nAWds3+ndsT2zO etamustudent@windemo

# on MacOS / Linux
$ cat .ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDdcar+W6ILPAXtBW6xfIH5cpVHwR2nAWds3+ndsT2zO etamustudent@windemo
```

The public key is just that, public information, so it is not problematic to see or give someone your public key. However the private key of a public / private key pair should be treated like a password. Once we add this key to GitHub, anyone with your private key would be able to do any task on your GitHub repositories (like modifying or deleting repositories) that you could do with this key.

To add a new key on GitHub, open up your GitHub account **Settings** (upper right hand side of GitHub, pull down your User navigation and select **Settings**). In your GitHub **Settings**, find your **SSH and GPG keys** settings. From here you should select the button to add a **New SSH key**.

Copy your public key from your file and paste it into the **Key**. You should give each of your keys a **Title** that will help you remember its purpose, for example "CSci 233 Application Development Key". Press the **Add SSH Key** to add this new key to your GitHub account.

IMPORTANT: Once you think you have successfully generated your `ssh` key and added it to your GitHub account, it is **imperative** that you test that your key allows you to access and authenticate with GitHub.

Open up a Terminal and authenticate your new default key with GitHub like this:

```
C:\Users\etamustudent> ssh git@github.com
The authenticity of host 'github.com (140.82.113.4)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
PTY allocation request failed on channel 0
Hi etamustudent! You've successfully authenticated, but GitHub does not provide shell access.
Connection to github.com closed.
```

Note: The first time you authenticate to a new host using an `ssh` key, it is normal to get the warning message about the 'authenticity of host' not being known. You should answer `yes` to add `github.com` to the list of known hosts you are using.

Note: Sometimes you may need to hit a Return or 2 after answering `yes` before the authentication will complete and you will get your message.

The key should successfully authenticate, as shown above. If instead you see:

```
C:\Users\etamustudent> ssh git@github.com
git@github.com: Permission denied (publickey).
```

Then either you did not correctly get your public key copied and added into your GitHub account. Or you did not generate your key correctly, or the key has gone missing. Check again your GitHub ssh key settings, and if needed delete any current key files and public key files, regenerate a new key, and add the new key to GitHub. Do not move on from this step until you can successfully authenticate on the command line with `github.com`. Get help from a GA or teacher if you have problems authenticating your GitHub key.

Text Editors / IDE for Python Programming Assignments

In addition to a Python 3 interpreter and `git` / `ssh` tools, you will need a programming text editor. You could use anything you are familiar with to edit and create Python code. Basic editors that have versions on Windows and Mac include `Notepad++`, `Sublime Text` and `TextMate`.

If you don't have a preference or are unsure, we recommend installing and using Download Visual Studio Code as your editor. In this section we give some notes on setting it up in a Windows environment to edit and run your class assignments. The link given should allow you to download the appropriate installers for Windows, MacOS or Linux. And most of the following instructions should apply, though some may be specific to setting up in a Windows environment.

Once installed, open `VSCode` and go to the **Extensions** (left side bar). Search for "Python" in the extensions marketplace. There will be many useful extensions here for Python development. Find the one simply named "Python" published by "Microsoft (microsoft.com)". This is the official extension pack, and if you select and install it, it will install "Python", "Python Debugger" "Pylance" and "Python Environments" extensions, all of which you might find useful to have for your assignments.

Note: The "Python Environments" extension can be helpful for finding and managing different Python distributions you might have available, and for setting up Python virtual environments for projects. If you don't see a "Python" icon on your left hand side bar in VSCode after installing this extension, it may not be enabled. As of the writing of this guide it is not enabled by default. You should open up your VSCode **Settings** and search for 'python.useEnvironmentsExtension'. Enable it by checking on the checkbox. Once enabled you should now see the "Python" extension in your left hand VSCode sidebar. You can use this to check that the installed Python interpreter is available, e.g. the "Global" python environment, among other things.

References / Additional Resources

- How to open a Command Prompt in Windows 10, 11
- Download Python from python.org
- Anaconda Python Distribution
- Homebrew
- How to use the Terminal command line in MacOS
- Installing Python 3 on Mac OS X
- Git - Install for Windows
- Git - Install for MacOS
- Customizing Git - Git Configuration
- GitHub.com
- Download Visual Studio Code